

**SIMATS ENGINEERING**

**CO2 ASSESSMENT TOOL 2**

**HITESH C**

**192524005**

Course Code: CSA1401

Course Name: Compiler Design

**Q1. Token stream: id + id \* id — how does the parser validate syntactic structure?**

**Answer:** The parser takes the token stream from the lexical analyzer and attempts to build a derivation (and, equivalently, a parse tree) that reduces the tokens to the grammar's start symbol using the production rules of the CFG, for example  $E \rightarrow E + E \mid E * E \mid \text{id}$ . Validity is checked by confirming that every token can be matched against some right-hand side at the correct point in the derivation; if the parser also applies operator precedence (here,  $*$  binding tighter than  $+$ ), the resulting tree groups the multiplication before the addition —  $\text{id} + (\text{id} * \text{id})$ . If no sequence of productions can account for the token stream, the parser reports a syntax error instead of accepting it.

**Q2.  $S \rightarrow aSb \mid \epsilon$  — does 'aaabbb' belong to the language?**

$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaasbbb \Rightarrow aaas\epsilon bbb = aaabbb$

**Answer:** Yes. The derivation above (applying  $S \rightarrow aSb$  three times and then  $S \rightarrow \epsilon$ ) produces exactly 'aaabbb', so the string belongs to the language  $L = \{ a^n b^n \mid n \geq 0 \}$  generated by this grammar (here  $n = 3$ ).

**Q3. Remove left recursion from  $E \rightarrow E + T \mid T$** 

$E \rightarrow T E'$   
 $E' \rightarrow + T E' \mid \epsilon$

**Answer:** Using the standard transformation for immediate left recursion ( $A \rightarrow A\alpha \mid \beta$  becomes  $A \rightarrow \beta A'$ ,  $A' \rightarrow \alpha A' \mid \epsilon$ ) with  $A = E$ ,  $\alpha = "+ T"$  and  $\beta = "T"$  gives the equivalent right-recursive grammar shown above.

**Q4. FIRST(A) for  $A \rightarrow aB \mid \epsilon$ ,  $B \rightarrow b$** 

**Answer:**  $A \rightarrow a$  contributes the terminal 'a';  $A \rightarrow \epsilon$  contributes  $\epsilon$ . Hence  $\text{FIRST}(A) = \{ a, \epsilon \}$ .

**Q5. Is  $A \rightarrow Aa \mid b$  suitable for top-down parsing?**

**Answer:** No, not directly. The production  $A \rightarrow Aa$  is immediately left-recursive, which causes a top-down (recursive-descent / predictive) parser to loop forever expanding  $A$  without ever consuming input. The grammar must first be transformed by eliminating left recursion:  $A \rightarrow bA'$ ,  $A' \rightarrow aA' \mid \epsilon$ , which is equivalent and safely usable for top-down parsing.

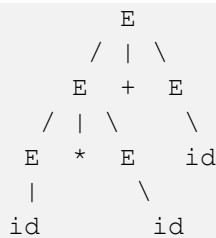
#### Q6. Left factor: $S \rightarrow \text{while } E \text{ do } S \mid \text{while } E \text{ do } S \text{ else } S$

```
S → while E do S S'
S' → else S | ε
```

**Answer:** Both alternatives share the common prefix “while E do S”, so it is factored out; the remaining choice (whether an 'else S' follows) is pushed into the new non-terminal S', which resolves the dangling-else style ambiguity for predictive parsing.

#### Q7. Parse tree for $\text{id} * \text{id} + \text{id}$ using $E \rightarrow E + E \mid E * E \mid \text{id}$

Applying the conventional precedence (\* before +), the expression groups as  $(\text{id} * \text{id}) + \text{id}$ :



#### Q8. Recursive descent procedures for $S \rightarrow aA$ , $A \rightarrow bA \mid c$

```
procedure S():
    match('a')
    A()

procedure A():
    if lookahead == 'b':
        match('b')
        A()
    else if lookahead == 'c':
        match('c')
    else:
        error("expected 'b' or 'c'")
```

#### Q9. Predictive parsing table for $S \rightarrow AB$ , $A \rightarrow a \mid \epsilon$ , $B \rightarrow b$

$\text{FIRST}(A) = \{a, \epsilon\}$ ,  $\text{FIRST}(B) = \{b\}$ ,  $\text{FOLLOW}(A) = \text{FIRST}(B) = \{b\}$ , so  $\text{FIRST}(S) = \{a, b\}$ .

| Non-terminal | a                  | b                        | \$ |
|--------------|--------------------|--------------------------|----|
| S            | $S \rightarrow AB$ | $S \rightarrow AB$       | —  |
| A            | $A \rightarrow a$  | $A \rightarrow \epsilon$ | —  |
| B            | —                  | $B \rightarrow b$        | —  |

**Q10. First three shift-reduce actions for  $\text{id} + \text{id}$** 

| Step | Stack | Input      | Action                           |
|------|-------|------------|----------------------------------|
| 1    | \$    | id + id \$ | Shift id                         |
| 2    | \$ id | + id \$    | Reduce $\text{id} \rightarrow E$ |
| 3    | \$ E  | + id \$    | Shift +                          |

**Q11. Identify the handle in  $\text{id} * \text{id} + \text{id}$  ( $E \rightarrow E + E \mid E * E \mid \text{id}$ )**

**Answer:** Scanning left to right, the first handle recognized is the leftmost 'id' (reduced by  $E \rightarrow \text{id}$ ). After both operands of the multiplication are individually reduced to E, the substring 'E \* E' (originating from 'id \* id') becomes the next handle and is reduced by  $E \rightarrow E * E$  — this reduction must occur before the '+' is processed because \* has higher precedence, so the overall first significant handle in the expression is 'id \* id'. Finally, once the trailing 'id' is reduced to E, the whole 'E + E' becomes the last handle, reduced by  $E \rightarrow E + E$ .

**Q12. Operator precedence relations for +, \* (with \* higher than +)**

|    | +         | *         | \$        |
|----|-----------|-----------|-----------|
| +  | $\cdot >$ | $< \cdot$ | $\cdot >$ |
| *  | $\cdot >$ | $\cdot >$ | $\cdot >$ |
| \$ | $< \cdot$ | $< \cdot$ | —         |

**Answer:**  $+ < \cdot * : '+'$  yields precedence to '\*', so a '\*' following a '+' is shifted.  $* \cdot > + : '*'$  takes precedence, so a completed \*-expression is reduced before a following '+' is processed. Equal operators ( $+ \cdot > +, * \cdot > *$ ) reduce due to left associativity.

**Q13. How operator precedence parsing handles  $\text{id} + \text{id} * \text{id}$** 

**Answer:** The parser shifts the first id and reduces it to an operand, then sees '+'. Since '+' has lower precedence than the '\*' that follows, the relation  $+ < \cdot *$  tells the parser to shift past '+' and keep shifting the second id and the '\*' rather than reducing immediately. Only when the second '\*' operand (the third id) is shifted and the next symbol (\$) shows  $* \cdot > \$$  does the parser reduce 'id \* id' to E first; then, with the relation  $+ \cdot > \$$ , it reduces the outer 'id + E' to the final E. The net effect is that multiplication is completed before addition, correctly parsing the expression as  $\text{id} + (\text{id} * \text{id})$ .

**Q14. What are LR parsers? Illustrate with a simple scenario.**

**Answer:** LR parsers are bottom-up, shift-reduce parsers that scan the input Left to right and construct a Rightmost derivation in reverse, using a stack together with deterministic ACTION and GOTO tables built from an automaton of LR items (LR(0), SLR(1), LALR(1), or canonical LR(1)). They can handle a much larger class of grammars than top-down parsers, including many left-recursive and ambiguous-looking grammars, with linear-time parsing and immediate, precise error detection.

Simple scenario — grammar  $S \rightarrow ( S ) \mid \epsilon$ , input "()": the parser shifts '(', recognizes that the empty string matches  $S \rightarrow \epsilon$  and reduces (pushing S), shifts ')', and finally reduces the completed handle (S) using  $S \rightarrow (S)$ ; the stack returns to just S and the input is exhausted, so "()" is accepted.

**Q15. Steps to construct an SLR parsing table for  $S \rightarrow AA, A \rightarrow aA \mid b$**

1. Augment the grammar with a new start symbol:  $S' \rightarrow S$ .
2. Construct the canonical collection of LR(0) item sets using closure and goto operations, starting from  $\text{closure}(\{S' \rightarrow \cdot S\})$ .
3. Compute the FIRST and FOLLOW sets of all non-terminals (needed to decide reduce actions).
4. Build the ACTION table: for a state containing  $[A \rightarrow \alpha \cdot a\beta]$ , enter a shift on terminal a; for a state containing a completed item  $[A \rightarrow \alpha \cdot]$  ( $A \neq S'$ ), enter 'reduce by  $A \rightarrow \alpha$ ' on every terminal in  $\text{FOLLOW}(A)$ ; for  $[S' \rightarrow S \cdot]$ , enter 'accept' on \$.
5. Build the GOTO table: for each state and non-terminal, record the state reached via goto.
6. Check every cell for shift/reduce or reduce/reduce conflicts; if none exist, the grammar is SLR(1) and the table is complete.

**Q16. FOLLOW(A) for  $S \rightarrow AB, A \rightarrow a \mid \epsilon, B \rightarrow b$**

**Answer:** In  $S \rightarrow AB$ , A is immediately followed by B, and B cannot derive  $\epsilon$ , so  $\text{FOLLOW}(A) = \text{FIRST}(B) = \{b\}$ .

**Q17. Augmented grammar for  $S \rightarrow aA, A \rightarrow b$** 

```
S' → S
S  → a A
A  → b
```

**Q18. Closure of the LR(0) item  $S' \rightarrow \cdot S$  for  $S \rightarrow aA, A \rightarrow b$** 

**Answer:** Closure( $\{[S' \rightarrow \cdot S]\}$ ) adds every production of S (since the dot is immediately before S):  $[S \rightarrow \cdot aA]$ . No further items are added because the symbol after the dot in this new item is the terminal 'a', not a non-terminal. Hence Closure =  $\{ [S' \rightarrow \cdot S], [S \rightarrow \cdot aA] \}$ .

**Q19. Is  $E \rightarrow E + E \mid id$  ambiguous? Check with  $id + id + id$** 

**Answer:** Yes, the grammar is ambiguous. The string 'id + id + id' has two distinct parse trees / leftmost derivations: one grouping as  $(id + id) + E \rightarrow E + E$  on the left first, and another grouping as  $id + (id + id) \rightarrow E + E$  on the right first. Since a single string admits more than one parse tree,  $E \rightarrow E + E \mid id$  is ambiguous (associativity is left undetermined by the grammar itself).

**Q20. Leftmost derivation for  $id + id * id$  using  $E \rightarrow E + E \mid E * E \mid id$** 

```
E ⇒ E + E
  ⇒ id + E
  ⇒ id + E * E
  ⇒ id + id * E
  ⇒ id + id * id
```

**Q21. Left factor  $S \rightarrow aBc \mid aBd \mid b$** 

```
S  → a B S' | b
S' → c | d
```

**Answer:** The first two alternatives share the common prefix 'aB', which is factored out into  $S \rightarrow aBS'$ , leaving S' to select between the differing suffixes 'c' and 'd'.

**Q22. FIRST(F) for  $F \rightarrow (E) \mid id$** 

**Answer:** FIRST(F) =  $\{ (, id \}$ , taken directly from the first terminal of each alternative.

**Q23. Is  $S \rightarrow aA \mid aB$  ( $A \rightarrow b, B \rightarrow c$ ) suitable for predictive parsing? Justify.**

**Answer:** Not directly. Both alternatives of  $S$  begin with the same terminal 'a' ( $\text{FIRST}(aA) = \{a\} = \text{FIRST}(aB)$ ), producing a FIRST/FIRST conflict: on lookahead 'a' the predictive parser cannot decide which alternative to expand. The grammar must first be left-factored, e.g.  $S \rightarrow aS', S' \rightarrow A \mid B$ , after which  $\text{FIRST}(A) = \{b\}$  and  $\text{FIRST}(B) = \{c\}$  are disjoint and predictive parsing becomes possible.

**Q24. Recursive descent parser procedures for  $S \rightarrow abS \mid \epsilon$**

```
procedure S():
    if lookahead == 'a':
        match('a')
        match('b')
        S()
    else:
        return          //  $\epsilon$ -production, valid when lookahead  $\in \text{FOLLOW}(S)$ 
```

**Q25. Predictive parsing table for  $S \rightarrow aS \mid b$  (terminals a, b, \$)**

$\text{FIRST}(aS) = \{a\}$ ,  $\text{FIRST}(b) = \{b\}$ ; since neither alternative derives  $\epsilon$ ,  $\text{FOLLOW}(S)$  is not needed for any table entry, and the \$ column has no valid entry (an input of just \$ cannot be derived from  $S$ ).

| Non-terminal | a                  | b                 | \$        |
|--------------|--------------------|-------------------|-----------|
| S            | $S \rightarrow aS$ | $S \rightarrow b$ | — (error) |